ME -- Kinematics and Dynamics of Machinery

Kinematics and Dynamics of Machinery Project

Simulation of Four-Bar and Crank-Slider Mechanisms

Code Explanation Report

Name	Roll Number
Mohamed Shaheem	ME24BTECH11042
Sahasra Reddy	ME24BTECH11051
Ajumeera Yashwanth Naik	ME24BTECH11005
Sai Nirali	ME24BTECH11011

GitHub Repository:

https://github.com/PLACEHOLDER

IIT Hyderabad | 2024–25

Contents

1 Libraries Used **2**
- 1.1 SFML — Simple and Fast Multimedia Library . . . 2
- 1.2 Dear ImGui . . . 2

2 Four-Bar Mechanism **2**
- 2.1 Data Structures . . . 2
- 2.2 Grashof Classification . . . 2
- 2.3 Position Analysis — `solvePositionBoth` and `solvePositionLive` . . . 3
- 2.4 Velocity Analysis . . . 3
- 2.5 Acceleration Analysis . . . 4
- 2.6 Coupler Point . . . 4
- 2.7 Simulation Loop . . . 4
- 2.8 Data Collection and Termination . . . 5

3 Four-Bar Edge Cases and Fixes **5**
- 3.1 EC-1 — `acos` Domain Overflow (NaN) . . . 5
- 3.2 EC-2 — Velocity Denominator Near Zero . . . 5
- 3.3 EC-3 — Ping-Pong Freeze at Toggle (Special Grashof) . . . 5
- 3.4 EC-4 — Double-Negation Bug (Previous Versions) . . . 6
- 3.5 EC-5 — Special Grashof Coupler Curve Incomplete . . . 6
- 3.6 EC-6 — NaN in Draw Calls . . . 6
- 3.7 Edge Case Summary . . . 7

4 Crank-Slider Mechanism **7**
- 4.1 Data Structure . . . 7
- 4.2 Position Analysis . . . 7
- 4.3 Velocity Analysis . . . 7
- 4.4 Acceleration Analysis . . . 8
- 4.5 Inversions . . . 8
- 4.6 Crank-Slider Edge Cases . . . 8

5 Common Patterns **9**
- 5.1 Main Loop Structure . . . 9
- 5.2 SFML y-Axis Negation . . . 9
- 5.3 Vector Arrow Drawing . . . 9

6 Quick Symbol Reference **9**

Libraries Used

SFML — Simple and Fast Multimedia Library

SFML is a C++ library for creating windows and drawing 2D graphics. In this project it handles everything visual — drawing lines (links), circles (joints), rectangles (slider block), and text labels. It follows a *game-loop* pattern: every frame the program clears the window, redraws everything from scratch, and displays it. Because the mechanism angles change each frame, the animation emerges naturally.

> **Key SFML types used:** `RenderWindow` (the application window), `Vertex` (a point with position and colour), `CircleShape` / `RectangleShape` (geometry), `Text` / `Font` (labels). The y-axis points **downward** in SFML, so all drawn y-coordinates negate the kinematic y-value: `O2.y - a*sin(theta2)`.

Dear ImGui

ImGui is an *immediate-mode* GUI library. Every widget (slider, button, text readout) is both drawn *and* read in a single function call each frame — no callbacks, no event listeners. `imgui-SFML` bridges it onto the SFML window.

> **Example:** `ImGui::SliderFloat("Theta2", &theta2_deg, 0, 360)` simultaneously renders the slider on screen and writes the dragged value into `theta2_deg`. One line does both.

Four-Bar Mechanism

Data Structures

```cpp
constexpr float dt = 0.001f;   // Euler integration time-step (seconds)

struct MechState {
    float theta2=0, theta3=0, theta4=0; // link angles (radians)
    float omega2=0, omega3=0, omega4=0; // angular velocities (rad/s)
    float alpha2=0, alpha3=0, alpha4=0; // angular accelerations (rad/s^2)
};
```

Links are numbered 1 (ground, fixed), 2 (input crank), 3 (coupler), 4 (output rocker). All angles are counter-clockwise from the positive x-axis. `dt = 0.001 s` means 1000 simulation steps per second of animation.

Grashof Classification

```cpp
static float grashofDelta(float* lengths) {
    // Sort to find S (shortest) and L (longest)
    // Returns S+L - (P+Q)
}
```

```
bool isStrictGrashof(float* l) { return grashofDelta(l) < -grashofTol(l); }
bool isSpecialGrashof(float* l){ return fabs(grashofDelta(l)) <= grashofTol(l); }
```

Physics: Grashof's condition states that at least one link can rotate fully if and only if $S + L \leq P + Q$, where S is the shortest link and L the longest. Three cases:

$S + L$ vs $P + Q$	Class	Simulation behaviour
$S + L < P + Q$	Class 1 (Grashof)	Crank rotates fully — use travel accumulation
$S + L = P + Q$	Class 2 (Special Grashof)	Change-point — use reversal counting
$S + L > P + Q$	Class 3 (Non-Grashof)	All links oscillate — use reversal counting

A relative tolerance `grashofTol = 0.001 × (S+L)` is used instead of a fixed epsilon, so the classification scales correctly with link size.

Position Analysis — `solvePositionBoth` and `solvePositionLive`

```
bool solvePositionBoth(float a,float b,float c,float d,float theta2,
                       float& th3A,float& th4A,float& th3B,float& th4B)
{
    Vector2f A(a*cos(theta2), a*sin(theta2));
    Vector2f r = Vector2f(d,0) - A;
    float R = sqrt(r.x*r.x + r.y*r.y);
    if(R>(b+c) || R<fabs(b-c)) return false;  // toggle check
    float cosArg = (b*b+R*R-c*c)/(2*b*R);
    cosArg = max(-1.f, min(1.f, cosArg));      // EC-1: clamp
    float alpha = acos(cosArg);
    float base  = atan2(r.y, r.x);
    th3A = base+alpha;  th3B = base-alpha;     // two branches
    ...
}
```

Physics: Given θ_2, joint $A = (a\cos\theta_2, a\sin\theta_2)$. Vector $\mathbf{r} = O_4 - A$ has magnitude R. The cosine rule on triangle A-B-O_4 gives:

$$\alpha = \cos^{-1}\left(\frac{b^2 + R^2 - c^2}{2bR}\right), \quad \theta_3 = \text{base} \pm \alpha \tag{1}$$

The $\pm$ gives the two assembly branches. `solvePositionLive` always picks the branch whose angles are *closest* to the previous frame, preventing spontaneous branch flipping during animation.

Velocity Analysis

```
float denom = sin(s.theta3 - s.theta4);
if(fabs(denom) < 1e-4f) denom = (denom>=0 ? 1e-4f : -1e-4f); // EC-2
```

```
s.omega3 = (a*s.omega2/b) * (sin(s.theta4-s.theta2) / denom);
s.omega4 = (a*s.omega2/c) * (sin(s.theta2-s.theta3) / denom);
```

Physics: Differentiating the vector loop $\mathbf{a} + \mathbf{b} = \mathbf{d} + \mathbf{c}$ gives:

$$\omega_3 = \frac{a\omega_2}{b} \cdot \frac{\sin(\theta_4 - \theta_2)}{\sin(\theta_3 - \theta_4)}, \qquad \omega_4 = \frac{a\omega_2}{c} \cdot \frac{\sin(\theta_2 - \theta_3)}{\sin(\theta_3 - \theta_4)} \tag{2}$$

Acceleration Analysis

Differentiating the velocity equations yields a 2×2 linear system in α_3 and α_4, solved by Cramer's rule:

$$\alpha_3 = \frac{CD - AF}{AE - BD}, \qquad \alpha_4 = \frac{CE - BF}{AE - BD} \tag{3}$$

where $A, \ldots, F$ absorb all centripetal (ω^2) terms. If the determinant $AE - BD < 10^{-6}$ the update is skipped to avoid division by zero.

Coupler Point

```
float phi = couplerAngleDeg * PI / 180.f;
Vector2f rAP(couplerDist*cos(s.theta3+phi),
            -couplerDist*sin(s.theta3+phi));
Vector2f couplerPoint(Ap.x + rAP.x, Ap.y + rAP.y);
```

A point P fixed to the coupler at distance r and angle ϕ relative to θ_3. Its velocity is $\mathbf{V}_P = \mathbf{V}_A + \boldsymbol{\omega}_3 \times \mathbf{r}_{AP}$ and its acceleration adds centripetal and tangential terms.

Simulation Loop

```
MechState next = s;
next.omega2 += next.alpha2 * dt;  // Euler integration
next.theta2 += next.omega2 * dt;

if(solvePositionLive(a,b,c,d,next,alternateBranch,false))
{
    s = next;  // valid step -- commit
    solveVelocity(a,b,c,s);
    solveAcceleration(a,b,c,s);
}
else
{
    s.omega2 *= -1.f;  // toggle hit -- reverse direction
    // nudge out of toggle zone (see EC-3)
}
```

Physics: Forward Euler integration: $\omega_2^{n+1} = \omega_2^n + \alpha_2 \cdot dt$, $\theta_2^{n+1} = \theta_2^n + \omega_2^{n+1} \cdot dt$. The step is performed on a *copy* of the state first. Only if the position solver succeeds is the copy committed to the live state. This ensures θ_2 and θ_3 are always consistent.

Data Collection and Termination

```
float reqRev = requiredRevolutions(lengths);
// Strict Grashof: collect until travelAccum >= 2*PI (one revolution)
// Special Grashof + Non-Grashof: collect until reversalCount >= 2
```

Assumption: For true Grashof mechanisms the crank completes full revolutions, so one cycle = 2π radians of travel. For all other cases the mechanism oscillates, so one cycle = 2 direction reversals (go forward, hit toggle, go back, hit toggle).

Four-Bar Edge Cases and Fixes

EC-1 — `acos` Domain Overflow (NaN)

Bug: Near a toggle, floating-point arithmetic pushes the `acos` argument slightly outside $[-1, 1]$, producing `NaN` which corrupts all subsequent state.

Fix: Clamp the argument before calling `acos`.

```
float cosArg = (b*b+R*R-c*c)/(2*b*R);
cosArg = max(-1.f, min(1.f, cosArg));  // safe
float alpha = acos(cosArg);
```

EC-2 — Velocity Denominator Near Zero

Bug: When $\theta_3 \approx \theta_4$ (toggle), $\sin(\theta_3 - \theta_4) \approx 0$ causes division by zero, giving `inf` velocities.

Fix: Signed clamp preserves direction.

```
if(fabs(denom) < 1e-4f)
    denom = (denom >= 0.f ? 1e-4f : -1e-4f);
```

EC-3 — Ping-Pong Freeze at Toggle (Special Grashof)

Bug: After reversing direction, if the state is exactly on the toggle boundary, the next frame immediately fails again and re-reverses. The mechanism oscillates between two failed frames and never moves — it appears frozen.

Fix: After reversing, immediately nudge up to 200 micro-steps in the new direction until a valid position is found.

```
s.omega2 *= -1.f;  // reverse
for(int k = 0; k < 200; k++) {
    MechState nudge = s;
    nudge.theta2 += nudge.omega2 * dt;
    if(solvePositionLive(..., nudge, ...)) { s = nudge;
        break; }
```

```
}
```

EC-4 — Double-Negation Bug (Previous Versions)

Bug: Earlier versions used both `simDirection *= -1` and `omega2 *= -1`. The two negations cancelled out, so the nudge stepped in the same direction as before and could never escape the toggle.

Fix: Remove `simDirection` entirely. The sign of `omega2` alone controls direction. One negation = one reversal.

EC-5 — Special Grashof Coupler Curve Incomplete

Bug 1: `requiredRevolutions` returned `4*PI` for Special Grashof, but the crank cannot complete a full revolution — it oscillates. Collection never terminated.
Bug 2: The reversal counter was guarded by `!grashof`, but `isGrashof()` returns `true` for Special Grashof, so `reversalCount` never incremented.

Fix 1: Only strict Grashof uses travel accumulation. Special Grashof and Non-Grashof both use reversal counting.

```
float requiredRevolutions(float* lengths) {
    if(isStrictGrashof(lengths)) return 2.f*PI;
    return -1.f;  // reversal-based for everything else
}
```

Fix 2: Count reversals unconditionally.

```
if(collecting) reversalCount++;  // was: if(!grashof &&
    collecting)
```

EC-6 — NaN in Draw Calls

Bug: If a NaN position reaches SFML's draw call, it renders at undefined screen coordinates or crashes.

Fix: Wrap all draw calls in a finite-check guard.

```
auto safeDrawLine = [&](Vector2f p1, Vector2f p2, Color col=
    Color::White) {
    if(!isFiniteVec(p1) || !isFiniteVec(p2)) return;
    // ... draw
};
```

Edge Case Summary

Case	Root Cause	Fix
EC-1	`acos` argument $\notin [-1, 1]$	Clamp to valid domain
EC-2	Velocity denominator ≈ 0	Signed clamp
EC-3	Ping-pong freeze at toggle	Nudge loop after reversal
EC-4	Double-negation cancels reversal	Remove `simDirection`
EC-5	Wrong termination for Special Grashof	Strict Grashof only uses travel; rest use reversals
EC-6	NaN in SFML draw calls	`isFiniteVec` guard

Crank-Slider Mechanism

Data Structure

```
struct MechState {
    float theta2=0, theta3=0;  // crank and coupler angles
    float omega2=0, omega3=0;
    float alpha2=0, alpha3=0;
    float slider=0;            // x-position of slider along rail
    float sliderVel=0;
    float sliderAcc=0;
};
```

The slider replaces the output rocker. Its entire kinematic state is described by position, velocity, and acceleration along the horizontal rail.

Position Analysis

```
s.slider = a*cos(s.theta2) + sqrt(b*b - a*a*sin(s.theta2)*sin(s.theta2));
s.theta3 = atan2(-a*sin(s.theta2), s.slider - a*cos(s.theta2));
```

Physics: With O_2 at the origin and the rail along $y = 0$, joint $A = (a\cos\theta_2, a\sin\theta_2)$. Since B must lie on $y = 0$ and $|AB| = b$:

$$x_B = a\cos\theta_2 + \sqrt{b^2 - a^2\sin^2\theta_2} \tag{4}$$

Assumption: No slider offset — the rail passes through O_2.

Velocity Analysis

```
// Slider velocity from differentiating x_B:
s.sliderVel = -a*s.omega2*sin(s.theta2)
            - (a*a*s.omega2*sin(s.theta2)*cos(s.theta2)) / root;

```

```
// omega3 from y-constraint (B must have zero y-velocity):
s.omega3 = (a*s.omega2*cos(s.theta2)) / (s.slider - a*cos(s.
    theta2));
```

ω_3 comes from the constraint that $V_{By} = 0$: differentiating the y-component of B's position and setting it to zero gives ω_3 directly.

Acceleration Analysis

```
// y-equation (A_{By} = 0): solve for alpha3 first
s.alpha3 = (s.omega3*s.omega3*rBAy - AAy) / rBAx;

// x-equation: then get slider acceleration (MINUS sign is
    critical)
s.sliderAcc = AAx - s.alpha3*rBAy - s.omega3*s.omega3*rBAx;
```

Physics: The y-component of $\mathbf{A}_B$ must be zero (slider constrained to $y = 0$). This isolates α_3. The x-component then gives $\ddot{x}_B$.

> **Critical sign:** The centripetal term must be **subtracted**: `- omega3*omega3*rBAx`. A `+` sign breaks the closure of the acceleration vector triangle.

Inversions

Inv.	Fixed link	Type	Visual transform
1	Frame	Standard crank-slider	None
2	Crank (O_2–A)	Oscillating cylinder	Rail rotates by θ_2
3	Coupler (A–B)	—	Rail rotates by θ_3
4	Slider (B)	Rotary engine	Translate so B is fixed

The kinematic values are identical across all inversions. Only the screen coordinate transform changes — direction of travel and which point stays stationary on screen.

Crank-Slider Edge Cases

> **EC-A — Acceleration sign error:** Using `+` instead of `-` for the centripetal term in `sliderAcc` caused the acceleration triangle to not close. Fixed by correcting the sign.

> **EC-B — `sf::State` name clash:** Adding `using namespace sf` caused a conflict between the user-defined `struct State` and SFML's internal `sf::State` enum. Fixed by renaming the struct to `MechState`.

> **EC-C — `rBAy` out of scope:** `rBAy` was a local variable inside `solveAcceleration` but also needed in the acceleration diagram drawing block. Fixed by recomputing it locally in the drawing block.

Common Patterns

Main Loop Structure

Both programs follow the same pattern every frame: **(1) Poll events → (2) Update kinematics → (3) Clear window → (4) Draw → (5) Display**. The animation is simply the kinematics re-solved each frame as θ_2 increments.

SFML y-Axis Negation

All drawn y-coordinates negate the kinematic y-value because SFML's y-axis points downward while kinematics uses the standard upward convention:

```
Vector2f A(O2.x + a*cos(s.theta2),
           O2.y - a*sin(s.theta2));  // note the minus
```

Vector Arrow Drawing

Velocity and acceleration diagrams draw each vector as a line plus two arrowhead lines at ±30ř from the tip:

```
float ang = atan2(end.y-start.y, end.x-start.x);
Vector2f a1(end.x - size*cos(ang-PI/6), end.y - size*sin(ang-PI
    /6));
Vector2f a2(end.x - size*cos(ang+PI/6), end.y - size*sin(ang+PI
    /6));
```

All vectors are auto-scaled so the longest one is exactly 120 px (velocity) or 150 px (acceleration), making diagrams always fit the 500×500 window regardless of physical magnitudes.

Quick Symbol Reference

Symbol	Variable	Meaning
a, b, c, d	`a,b,c,d`	Link lengths (mm)
θ_2	`s.theta2`	Input crank angle (rad)
ω_2	`s.omega2`	Input angular velocity (rad/s)
α_2	`s.alpha2`	Input angular acceleration (rad/s^2)
x_B	`s.slider`	Slider position (mm)
`dt`	`dt`	Euler time-step = 0.001 s
S, L, P, Q	(sorted)	Grashof link lengths
`reversalCount`	`reversalCount`	Toggle crossings (termination)
`travelAccum`	`travelAccum`	Cumulative $\lvert\Delta\theta_2\rvert$ (rad)